

Distributed Systems & Algorithms Deep Dive

A reference covering consistent hashing internals, rate limiting in production, probabilistic data structures, consensus algorithms, logical clocks, and classic tree/graph algorithms.

Contents

1. [Consistent Hashing — Why Only Neighboring Keys Move](#)
2. [Token Bucket & Leaky Bucket in Industry](#)
3. [Bloom Filters — Real-World Applications](#)
4. [Merkle Trees — Real-World Applications](#)
5. [Raft — Leader Election Deep Dive](#)
6. [Distributed Logs — Redis Streams & Replication Log](#)
7. [Lamport Timestamps](#)
8. [Vector Clocks Deep Dive](#)
9. [Reservoir Sampling](#)
10. [HyperLogLog & Redis PFCOUNT](#)
11. [Byzantine Fault Tolerance — PBFT Deep Dive](#)
12. [Skip Lists & Redis Sorted Sets](#)

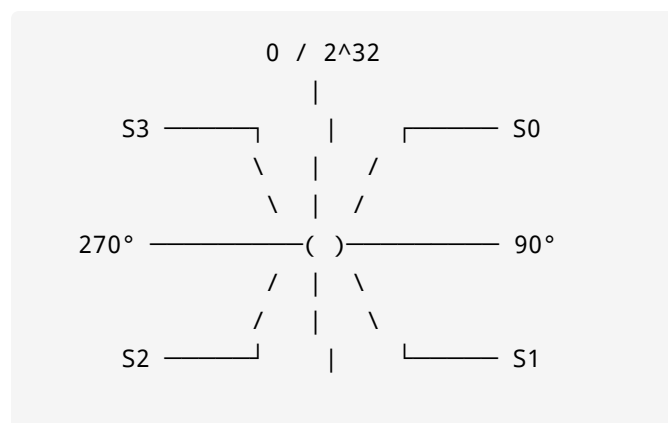
- 13. [AVL Trees vs. Red-Black Trees](#)
 - 14. [BFS & DFS – Graphs and Trees](#)
 - 15. [Kahn's Algorithm \(Topological Sort\)](#)
-

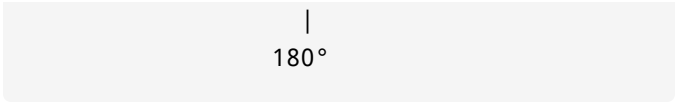
1. Consistent Hashing

What a circular hash space ("ring") actually is

A hash function like SHA-1 or MurmurHash produces outputs in a fixed range — say 0 to $2^{32} - 1$.

Consistent hashing treats that range as a **circle** instead of a line: conceptually, you take the number line from 0 to $2^{32} - 1$ and bend it around so that the maximum value connects back to 0. That's the entire trick — it's not a special data structure, just a decision to treat "one past the maximum hash value" as "back to zero," the same way a clock treats one second past 11:59:59 as 12:00:00.





180°

Both **servers** and **keys** are hashed with the same function, so both land on points around this circle. The ownership rule is simple: *a key belongs to the first server you reach walking clockwise from the key's position* (its "successor" on the ring).

Why adding/removing a server only remaps its immediate neighborhood

Think of the ring as being carved into arcs, one arc per server, running from "just after the previous server" to "the server itself." A key's owner is determined entirely by *which arc it falls into* — nothing else about the ring matters to that key.

Adding a server S_{new} : It lands somewhere on the ring, say between existing servers S_{prev} and S_{next} (its clockwise neighbor). Before the insertion, S_{next} owned the whole arc from S_{prev} to itself. After insertion, that arc is split in two: the portion between S_{prev} and S_{new} now belongs to S_{new} , and the portion between S_{new} and S_{next} still belongs to S_{next} . Only the keys in that first sub-arc move — everyone else on the ring still has the same clockwise-nearest server they always did, because their arc boundaries never changed.

Removing a server `S_old` : Its arc (from its predecessor to itself) has no owner anymore. The *only* boundary that changes is the one at `S_old` 's position — it disappears, so the arc merges into its clockwise neighbor `S_next` 's arc. `S_next` now owns `S_old` 's former keys in addition to its own. Every other server's arc is completely unaffected.

This is the crucial contrast with `hash(key) % N` : in modulo hashing, `N` is a single global parameter baked into every key's assignment, so changing `N` by even one ripples through almost the entire keyspace. In ring hashing, ownership is purely *local* — determined by two adjacent points — so a change at one point can only affect the arc between it and its nearest neighbor.

Virtual nodes don't change this principle; they just apply it at a finer grain. Each physical server is hashed to many points on the ring instead of one, so when a server joins or leaves, several small arcs move instead of one big one — which is precisely why virtual nodes also smooth out load distribution.

2. Token & Leaky Bucket in Industry

Token bucket is the more common of the two because it tolerates bursts, which matches how real traffic behaves:

- **Stripe** and **Amazon API Gateway** both throttle API requests using token-bucket-style limiters.
- **AWS EC2 "burstable" instances** (T-series) use a CPU-credit system that is a token bucket in spirit: credits accumulate during idle periods and are spent during CPU bursts.
- **Kubernetes** client libraries (`client-go`'s `flowcontrol` package, and Go's own `golang.org/x/time/rate`) implement token bucket for rate-limiting API server calls.
- **Linux traffic control** (`tc`) has a `tb` (token bucket filter) queuing discipline for shaping network bandwidth at the kernel level.

Leaky bucket shows up wherever a *fixed, smooth output rate* matters more than allowing bursts:

- **Shopify's** REST Admin API historically documented a leaky-bucket limiter.
- Traffic shaping in classic **ATM (Asynchronous Transfer Mode) networking** originally formalized the leaky bucket as a "traffic policing" mechanism — it predates its use in web APIs by decades.
- **GCRA (Generic Cell Rate Algorithm)**, used by **Cloudflare** and referenced in Stripe's public rate-limiter write-up, is mathematically a leaky bucket implemented without an actual queue (it tracks a "theoretical arrival time" instead), which makes it cheap to compute in Redis with a single key.

A rule of thumb: if the interviewer/system needs to *absorb spikes* (e.g., a user firing off 20 requests in one second, then going quiet), reach for token bucket. If the system needs to protect a downstream resource that can only be fed at a strictly constant rate, reach for leaky bucket.

3. Bloom Filters

Bloom filters answer "have I possibly seen this before?" with no false negatives and a tunable false-positive rate, using far less memory than a hash set. Real deployments:

- **Cassandra, HBase, and Bigtable-style LSM stores** attach a bloom filter to every on-disk SSTable so a `GET` for a nonexistent key can skip most files without a disk read.
- **Google Chrome's old Safe Browsing implementation** shipped a bloom filter of known-malicious URL hashes to the browser, so most benign URLs could be cleared locally without a network round-trip to Google.
- **Bitcoin SPV (lightweight) clients** use bloom filters (BIP 37) to ask full nodes "send me transactions that might involve these addresses" without revealing exactly which addresses they care about.

- **Medium's** recommendation pipeline reportedly uses bloom filters to avoid re-showing posts a user has already seen.
 - **Apache Spark** can build a bloom filter on a join key to skip shuffling rows that can't possibly match, speeding up large joins.
 - **Squid / CDN cache digests** let proxy servers exchange bloom filters of cached URLs so peers can decide whether to even ask each other for content.
-

4. Merkle Trees

Merkle trees make it cheap to detect *where* two large datasets differ without comparing them byte-for-byte:

- **Git** is fundamentally a Merkle DAG: every blob, tree, and commit is content-addressed by the hash of its contents (including the hashes of its children), which is why a single changed file produces a new hash all the way up to a new commit hash, and why `git` can cheaply tell two clones apart.
- **Bitcoin and Ethereum** put a Merkle root of all transactions in each block header, which lets "light" clients verify a specific transaction was included in a block by checking a short Merkle proof instead of downloading the whole block.

- **Cassandra, DynamoDB, and Riak** use Merkle trees for anti-entropy repair between replicas — comparing root hashes first, then walking down only the branches that disagree, so only the actually-divergent data gets transferred.
 - **Certificate Transparency (CT) logs** use Merkle trees to make an append-only log of issued TLS certificates tamper-evident and independently auditable.
 - **IPFS** addresses content via Merkle DAGs, which is also what allows deduplication — identical sub-trees across different files share the same hash and are stored once.
-

5. Raft — Leader Election

Raft splits consensus into leader election, log replication, and safety. Here's the election mechanism in full detail.

Node states and terms

Every node is a **Follower**, **Candidate**, or **Leader**. Time is divided into **terms** — monotonically increasing integers that act as a logical clock. At most one leader exists per term (possibly none, if an election fails to produce a winner).

The election trigger

Followers expect periodic **heartbeats** from the leader (an `AppendEntries` RPC, possibly carrying zero log entries). Each follower has an **election timeout**, randomized per node (the original paper uses 150–300 ms), that resets every time a valid heartbeat or vote arrives. If the timeout fires with no heartbeat, the follower assumes the leader is dead or unreachable and starts an election:

1. Increment its current term.
2. Transition to **Candidate**.
3. Vote for itself.
4. Send `RequestVote` RPCs, in parallel, to every other node — including its term number and information about how up-to-date its own log is.

Voting rules

A node grants its vote for a given term at most once (first-come-first-served), and only if:

- It hasn't already voted for someone else in this term, **and**
- The candidate's log is at least as up-to-date as the voter's own log (compared first by the term of the last log entry, then by log length). This is the **election-restriction** safety rule — it guarantees a candidate can only win if it holds all entries any

prior leader could have committed, which is what makes "leader completeness" hold later during log replication.

Winning, losing, and splitting

- If the candidate collects votes from a **majority** of the cluster ($\lfloor n/2 \rfloor + 1$), it becomes leader immediately and starts sending heartbeats to assert authority and suppress other elections.
- If it receives an `AppendEntries` from another node claiming leadership with an equal-or-higher term, it steps down to Follower.
- If neither happens before its own election timeout expires again, the term is likely **split** (votes divided across multiple candidates, none reaching a majority). The candidate simply increments its term and tries again. The **randomized** timeout is what makes split votes rare and self-resolving in practice — one node almost always times out and starts campaigning meaningfully before others, since timeouts are drawn independently.

Why this is safe

Because a term can have only one leader (majorities always overlap by at least one node, so two different candidates can't both win a majority in the same term), and because the election restriction ensures the winner's log is a superset of everything committed

so far, Raft guarantees that once an entry is committed, every future leader will already have it in its log.

Practical usage

- **etcd** (Kubernetes' backing store) and **HashiCorp Consul** both use Raft directly for their strongly-consistent metadata stores.
- **CockroachDB** and **TiKV** run one Raft group per data shard/range for replication.
- **Kafka's KRaft mode** replaced ZooKeeper-based coordination with a Raft-derived protocol for metadata consensus.
- **RabbitMQ quorum queues** use Raft for queue replication.
- **MongoDB's** replica-set election protocol is heavily Raft-inspired (though it predates and isn't identical to Raft).

6. Distributed Logs

A "distributed log" is an append-only, ordered sequence of records that multiple nodes replicate and consume — it's the same abstraction underneath Raft's replicated log, Kafka's partitions, and a database's write-ahead log (WAL). Redis gives two

concrete, very different implementations worth understanding.

6a. Redis replication backlog (the log behind master-replica sync)

Every Redis master keeps a **circular in-memory buffer** — the replication backlog — containing the most recent N bytes of the write command stream (the same stream that gets applied to the dataset). Each byte in that stream has a monotonically increasing **replication offset**. Every replica tracks the offset up to which it has applied commands.

When a replica connects (or reconnects after a network blip), it runs `PSYNC` with the master:

- If the replica's last known offset still exists inside the master's backlog window, the master performs a **partial resynchronization** — it just streams the missing bytes from the backlog, and the replica is caught up in milliseconds without re-transferring the whole dataset.
- If too much time has passed and the offset has fallen out of the backlog (the buffer is finite), the master falls back to a **full resync**: it forks, dumps an RDB snapshot, sends that, then streams new commands from that point on.

This is a genuinely distributed log algorithm: the backlog is the log, the offset is the log's index, and `PSYNC` is the "give me everything after offset X" read API — exactly the semantics Kafka consumers use against a partition, or a Raft follower uses to catch up on missing entries.

6b. Redis Streams (`XADD` / `XREAD`) — an explicit log data type

Streams are Redis's purpose-built log data structure, closer in spirit to a lightweight Kafka topic.

- **Entry IDs:** Each entry gets an ID of the form `<millisecondsTime>-<sequenceNumber>`, auto-generated to be strictly increasing, which gives the stream a total order for free.
- **Internal structure:** A stream is stored as a **radix tree** keyed by ID, whose leaves point to **listpacks** (compact, sequentially-encoded arrays of entries) — batching entries into listpacks avoids the overhead of one full tree node per record and keeps memory dense for a mostly-append workload.
- **Reading:** `XRANGE` / `XREVRANGE` do point/range queries by ID; `XREAD` supports blocking reads for "give me anything new after ID X," which is how consumers tail the log in near real time.

- **Consumer groups:** `XREADGROUP` lets multiple consumers **cooperatively split** a stream — Redis tracks, per group, which entries have been delivered but not yet acknowledged in a **Pending Entries List (PEL)**. A consumer calls `XACK` once it's done processing an entry; unacknowledged entries can be reclaimed by another consumer via `XCLAIM` / `XAUTOCLAIM` if the original one crashes — giving **at-least-once** delivery semantics, the same guarantee Kafka consumer groups provide.
- **Trimming:** `XADD ... MAXLEN ~ 1000` (or `XTRIM`) caps the log's size, since unlike a WAL you often don't want to retain history forever.

Practical usage patterns

- Event sourcing / activity feeds (append user actions, replay to rebuild state).
 - Lightweight task queues with multiple workers via consumer groups.
 - Chat message logs (the design in the earlier system-design summary uses exactly this shape — an append-only, ID-ordered log per conversation).
 - As a smaller-scale, lower-latency alternative to Kafka when you don't need Kafka's disk-based, multi-day retention or cross-datacenter mirroring.
-

7. Lamport Timestamps

Lamport timestamps give every event in a distributed system a number such that if event *A could have causally influenced* event B, A's timestamp is guaranteed to be smaller — establishing a **partial order** called "happens-before," without needing synchronized physical clocks.

The rules:

1. Each process keeps a local counter, starting at 0.
2. Before executing any event, a process increments its counter.
3. When sending a message, the process attaches its current counter value.
4. When receiving a message with timestamp t , the process sets its counter to $\max(\text{local_counter}, t) + 1$ before processing the message.

What it guarantees — and what it doesn't: if A happens-before B (there's a causal chain of events/messages connecting them), then $\text{timestamp}(A) < \text{timestamp}(B)$. But the converse isn't true — two *concurrent* events (neither influenced the other) can end up with timestamps in either order, or even the same value. To get a **total order** (useful for things like consistently ordering operations across replicas), you break ties with a static, unique process

ID appended to the timestamp: `(counter , process_id)` .

Practical relevance: Lamport clocks are mostly a conceptual/teaching tool today — production systems tend to reach for vector clocks (below) when they need to detect concurrent/conflicting writes, or for hybrid logical clocks / physical clocks with bounded uncertainty (like Google Spanner's TrueTime) when they need something closer to global ordering with real time semantics. But the happens-before relation Lamport formalized underpins the reasoning behind almost every causal-consistency and conflict-detection scheme in distributed databases.

8. Vector Clocks

Vector clocks extend Lamport's idea to let you detect concurrency explicitly, not just causal ordering — which is exactly what you need to know "did these two writes conflict, or did one clearly come after the other?"

Structure and rules

A vector clock for a system with N nodes is an array/map of N counters, one per node: `[S1: v1 , S2: v2 , ..., Sn: vn]` . Each data item carries its own vector clock.

- **On a local write at node S_i** : increment only S_i 's own counter in the vector.
- **On merging/reading two versions**: take the element-wise maximum of both vectors.

Comparing two vector clocks

Given clocks $V(A)$ and $V(B)$:

- **A happened-before B** if every entry in $V(A)$ is $\leq$ the corresponding entry in $V(B)$, and at least one is strictly less. (A is an ancestor of B — no conflict, B simply supersedes A.)
- **A and B are concurrent** (a genuine conflict) if neither dominates the other — i.e., some entry in $V(A)$ is greater than the corresponding entry in $V(B)$, and some entry in $V(B)$ is greater than the corresponding entry in $V(A)$.

Worked example

```
D1 written by  $S_x$            →  $D1[(S_x,1)]$ 
D2 = update of D1 by  $S_x$  →  $D2[(S_x,2)]$ 
(descends from D1, no conflict)
D3 = update of D2 by  $S_y$  →  $D3[(S_x,2),$ 
 $(S_y,1)]$ 
D4 = update of D2 by  $S_z$  →  $D4[(S_x,2),$ 
 $(S_z,1)]$   (D3 and D4 are concurrent –
conflict!)
```

Neither `D3`'s nor `D4`'s vector dominates the other, so a client or the storage system must reconcile them (e.g., merge, or ask the application/user to pick a winner) — this is exactly the mechanism behind **Amazon Dynamo** and **Riak**'s "sibling values" that clients sometimes see on read.

Practical caveat and mitigation

Vector clocks grow with the number of *distinct nodes that have ever written* a given key, which is unbounded in principle. Two mitigations used in practice: (1) capping the vector's length and evicting the oldest `[node, counter]` entries once a threshold is hit (Dynamo's approach — a pragmatic tradeoff that can occasionally cause an incorrect ancestor determination, which Amazon's own paper reports as rare enough in practice to be acceptable), and (2) **dotted version vectors**, a refinement used in later Riak versions that avoids a lot of the "false conflict" noise plain vector clocks produce under concurrent writes from the same client.

9. Reservoir Sampling

Reservoir sampling solves: given a stream of items whose total length `n` is unknown (or too large to

store), pick k items uniformly at random using only $O(k)$ memory, in a single pass.

Algorithm R (the classic version)

```
reservoir = first k items of the stream
for i = k+1, k+2, ... (each subsequent
item, 1-indexed):
    j = random integer in [1, i]
    if j <= k:
        reservoir[j] = item_i    # replace
an existing sample
```

Why this produces a uniform sample

By induction: after processing the first k items, every item is in the reservoir with probability 1 (trivially uniform). When item i (for $i > k$) arrives, it enters the reservoir with probability k/i . For an item that's already in the reservoir to *survive* this round, it must **not** be the one selected for replacement, which happens with probability $1 - (1/i) \cdot (k/i)$... more directly: the standard proof shows that after processing all n items, every item independently ends up in the final reservoir with probability exactly k/n , regardless of when it arrived in the stream. The intuition: each new item gets a "fair shot" at k/i of entering, and each existing item is displaced with exactly the probability needed to keep everyone's long-run odds equal.

The $k = 1$ special case

Keep item 1 as the candidate; for each subsequent item i , replace the candidate with probability $1/i$. This is the same algorithm with $k=1$.

A faster variant: Algorithm L

Algorithm R does one random draw per incoming item, which is wasteful once the reservoir is much smaller than the stream (most draws don't cause a replacement). **Algorithm L** instead computes, using an exponential/geometric random variable, exactly *how many items to skip* before the next replacement – jumping ahead directly to the next item that will actually be selected. This reduces expected running time from $O(n)$ random draws to $O(k \cdot (1 + \log(n/k)))$, which matters a lot when streaming through billions of records.

Weighted variants

When items have different selection weights (not uniform), algorithms like **A-Res** or **A-Chao** generalize reservoir sampling by assigning each item a priority derived from `weight` and a random draw, keeping the top- k priorities seen so far.

Practical uses

- **Distributed tracing** (Google Dapper, Twitter/OpenZipkin) samples a fixed number of traces per time window without knowing in advance how many requests will occur.
 - **Database query optimizers** (e.g., collecting table statistics for query planning) sample rows from tables far too large to scan fully.
 - **Log/analytics pipelines** (e.g., sampling a fixed-size representative subset of a firehose of events for downstream ML training or debugging) when the firehose's total size isn't known ahead of time.
-

10. HyperLogLog

HyperLogLog (HLL) estimates the number of **distinct elements** in a multiset using a small, fixed amount of memory — trading exactness for massive space savings (kilobytes instead of storing every unique element).

The core idea (building up from Flajolet–Martin)

Hash every incoming item to a uniformly distributed bit string. In a good hash, the probability of seeing k leading zero bits is 2^{-k} . So if, across a stream, the *maximum* number of leading zeros you've observed in any hash is R , that's a signal the stream probably

contains on the order of 2^R distinct items — because you'd need roughly that many independent draws before a k -leading-zero pattern that rare shows up at all. That single-counter version (Flajolet–Martin) is noisy, though — HyperLogLog fixes the noise with two refinements.

Stochastic averaging (the "Hyper" part)

Instead of one counter for the whole stream, split the hash into two parts:

- The first b bits select one of $m = 2^b$ **registers** (buckets) — this is "stochastic averaging": every incoming item is randomly routed to one of m independent estimators instead of all of them updating a single counter.
- The remaining bits are used to compute the **rank** — the position of the leftmost 1-bit (i.e., leading-zero-count + 1) — and each register keeps only the **maximum rank** it has ever seen among items routed to it.

The estimator

Once all m registers hold their max-rank values $M[1..m]$, the cardinality estimate is a **harmonic mean** of $2^{M[j]}$ across registers, scaled by a bias-correction constant:

$$E = \alpha_m \cdot m^2 / (\sum 2^{(-M[j])} \text{ for } j = 1..m)$$

α_m is a constant (derived analytically, close to $0.7213 / (1 + 1.079/m)$ for large m) that corrects a known small-sample bias in the harmonic-mean estimator. The harmonic mean is used specifically because it's much less sensitive to a single register having an unusually large max-rank than an arithmetic mean would be. The standard error of the whole estimate works out to about $1.04 / \sqrt{m}$ — e.g., with $m = 16384$ registers, error is roughly 0.81%, regardless of whether you're counting thousands or billions of distinct items.

How Redis's `PFADD` / `PFCOUNT` / `PFMERGE` use this

- Redis uses $m = 2^{14} = 16384$ registers.
- Each register only needs about **6 bits** (enough to represent max-ranks up to ~50-something, far beyond what's ever realistically observed), so the dense encoding is $16384 \times 6 \text{ bits} \approx 12,288 \text{ bytes}$ — **about 12 KB**, regardless of whether you've added a thousand items or a billion. This is the number Redis's own documentation famously cites.
- For low cardinalities, Redis uses a **sparse encoding** (run-length-style compression of

mostly-zero registers) that's far smaller than 12 KB, and automatically promotes to the dense representation once enough registers become non-trivial.

- `PFADD key element` hashes the element, updates the appropriate register if the new rank is larger than what's stored.
- `PFCOUNT key` runs the harmonic-mean estimator over the registers.
- `PFMERGE dest src1 src2 ...` combines multiple HLLs by taking the **element-wise maximum** of their registers — which works correctly precisely because "max rank seen" is exactly what a register represents; merging never loses information the way summing would. This is what lets you maintain, say, a per-day unique-visitor HLL and cheaply merge seven of them into a correct weekly-unique-visitor estimate without re-processing any raw events.

Practical use elsewhere

Redis's unique-visitor counting is the textbook example, but the same technique (often literally the same algorithm) powers `APPROX_COUNT_DISTINCT` in **Google BigQuery** (where HLL was originally productionized), **Amazon Redshift's** `APPROXIMATE COUNT DISTINCT`, Elasticsearch's `cardinality` aggregation, and Presto/Trino's `approx_distinct`.

11. Byzantine Fault Tolerance

The problem

Crash-fault-tolerant protocols like Raft and Paxos assume a failed node simply stops responding.

Byzantine faults are worse: a faulty node might send *contradictory* or *malicious* messages — telling different peers different things, or lying outright. This matters whenever you can't fully trust every participant (blockchains, cross-organization systems) or want resilience against arbitrary software bugs/corruption, not just crashes.

The fundamental bound

Lamport, Shostak, and Pease's original "Byzantine Generals" result: to reach agreement while tolerating f Byzantine (arbitrarily faulty) nodes, you need at least $n \geq 3f + 1$ total nodes. Intuitively, if you only had $3f$, a faulty minority could split the honest majority's view of the world into two groups each of size $f + \text{honest-remainder}$, making it impossible for honest nodes to tell which "camp" reflects the true state.

PBFT (Practical Byzantine Fault Tolerance) — deep dive

PBFT (Castro & Liskov, 1999) was the first BFT protocol efficient enough for real systems (as opposed to purely theoretical). It runs in **views**, each with a designated **primary**, cycling through replicas in round-robin order.

Normal-case operation (3 phases, plus reply):

1. **Pre-Prepare:** A client sends a request to the primary. The primary assigns it the next sequence number and broadcasts a `PRE-PREPARE(view, seq, digest)` message to all backup replicas.
2. **Prepare:** Each backup, on accepting the pre-prepare (checking the view/sequence number are consistent with what it's seen before), broadcasts `PREPARE(view, seq, digest, replica_id)` to *every other replica* — not just the primary. Once a replica has collected $2f$ matching `PREPARE` messages (plus its own pre-prepare/prepare), it considers the request "**prepared**." This all-to-all broadcast is what lets replicas detect a lying primary — if the primary sent different requests to different replicas, the digests won't match.
3. **Commit:** Once prepared, each replica broadcasts `COMMIT(view, seq, digest, replica_id)` to all others. Once a replica collects $2f + 1$ matching commits (a quorum out of $3f + 1$ total nodes), the request is "**committed-local**" — it's now safe to execute, because *any* two quorums of size $2f+1$

out of $3f+1$ nodes must overlap in at least one **honest** node, which is what prevents two conflicting requests from both reaching commit.

4. **Reply:** Each replica executes the request against its local state machine and replies directly to the client. The client accepts the result once it has $f + 1$ matching replies – enough to guarantee at least one honest replica is among them.

View changes: If backups suspect the primary is faulty or unresponsive (a timer expires without progress), they broadcast a **VIEW-CHANGE** message and move to the next view, whose primary is the next replica in the rotation ($\text{view} \bmod n$). This is conceptually similar to Raft's election-on-timeout, but more involved: the new primary must prove, using certificates gathered from $2f+1$ replicas, that it is safely resuming from a state no committed request will be lost or reordered.

Cost: PBFT's normal-case messaging is $O(n^2)$ (the all-to-all Prepare/Commit broadcasts), which is why it doesn't scale to huge validator sets without modification.

Practical usage

- **Hyperledger Fabric** offers a PBFT-derived ordering service option for permissioned blockchain networks.

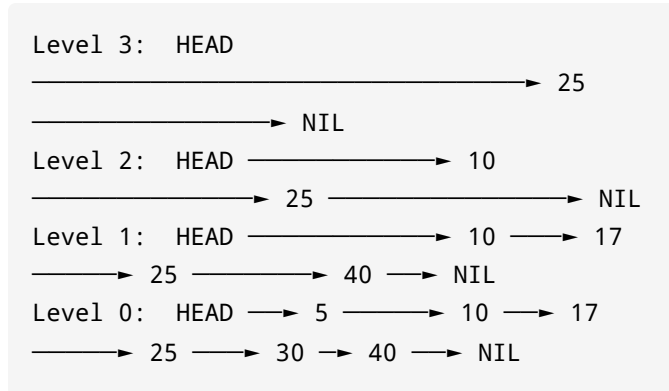
- **Tendermint** (used by the Cosmos ecosystem) implements a PBFT-style, round-based BFT consensus with slightly different message flow (propose/prevote/precommit).
 - **Diem/Libra's HotStuff** consensus is a more modern, *linear*-communication-complexity BFT protocol (each phase requires $O(n)$ messages instead of $O(n^2)$) explicitly designed to fix PBFT's scalability limits.
 - **Stellar Consensus Protocol** takes a different approach entirely (Federated Byzantine Agreement, based on overlapping "quorum slices" chosen by each node rather than one fixed global quorum).
-

12. Skip Lists

Exact structure

A skip list is a **probabilistic, multi-level linked list**. The base level (level 0) contains every element in sorted order, just like an ordinary linked list. Each element is independently "promoted" to level 1 with some fixed probability p (commonly $1/2$ or $1/4$); anything promoted to level 1 has an independent chance p of also being promoted to level 2, and so on. Each node stores an **array of forward pointers**, one per level it

participates in — a node present at level L has $L+1$ forward pointers, each pointing to the next node that also exists at that level.



Search starts at the head node's highest level and moves forward as long as the next node's key is still less than the target; when the next node's key would overshoot (or is `NIL`), the search drops down one level and repeats, until it reaches level 0 and lands on (or just before) the target. Because roughly $1/p$ nodes exist at each level up from the one below it, the expected number of levels is $O(\log n)$, and expected search/insert/delete time is $O(\log n)$ — the same asymptotic complexity as a balanced tree, but with much simpler code (no rotations or rebalancing, just randomized promotion at insert time).

Redis's sorted set (ZSET) implementation

For sets past a small-size threshold, Redis backs a sorted set with **two structures working together**:

- A **hash table** (`dict`) mapping `member` $\rightarrow$ `score` , giving $O(1)$ score lookups (`ZSCORE`).
- A **skip list** ordering elements by (`score` , `member`) — score primary, member as a lexicographic tiebreaker — giving $O(\log N)$ range operations (`ZRANGE` , `ZRANGEBYSCORE` , `ZRANK`).

Redis's skip list adds two refinements beyond the textbook version:

- **Span**: each forward pointer also stores how many level-0 nodes it "skips over." Summing spans while traversing lets Redis compute a node's **rank** in $O(\log N)$ — essentially turning it into an order-statistics skip list — which is what makes `ZRANK` / `ZRANGE BY RANK` efficient.
- **Backward pointers** at level 0 support efficient reverse traversal for `ZREVRANGE` .

Redis uses $p = 1/4$ (rather than the textbook $1/2$) and caps the max level at 32, a tuning choice that reduces average pointer overhead per node while still comfortably supporting skip lists with billions of elements. For genuinely small sorted sets, Redis skips all of this and just uses a compact **listpack** (a flat, sequentially-encoded array) instead — the skip list only kicks in once the set exceeds `zset-max-`

`listpack-entries` / `zset-max-listpack-value` thresholds, since scanning a small flat array is faster in practice than traversing pointers for tiny N.

13. AVL Trees vs. Red-Black Trees

Both are self-balancing binary search trees guaranteeing $O(\log n)$ search/insert/delete, but they optimize for different things.

AVL trees

Each node tracks a **balance factor** — the height of its left subtree minus the height of its right subtree — which must always be `-1`, `0`, or `1`. Any insert/delete that pushes a node's balance factor outside that range triggers **rotations** (single or double) to restore it, potentially cascading up toward the root. This keeps AVL trees **more rigidly balanced** than red-black trees (height is provably at most about $1.44 \log_2(n)$), which makes lookups slightly faster on average — but insertions/deletions can require $O(\log n)$ rotations in the worst case as the rebalancing cascades upward.

Red-black trees

Each node is colored red or black, subject to four invariants: the root is black; red nodes never have red

children (no two reds in a row on any path); every path from a node to any of its descendant null leaves passes through the same number of black nodes; and null leaves are considered black. These rules together guarantee the longest root-to-leaf path is at most **twice** the length of the shortest one — a looser balance guarantee than AVL, but one that can always be restored with **at most a constant number of rotations** (plus some recoloring) per insert/delete, no cascading required.

Intuition and tradeoffs

- AVL: tighter balance → faster reads, more expensive writes. Good for **read-heavy** workloads (e.g., databases doing far more lookups than modifications).
- Red-Black: looser balance → slightly slower reads, cheaper writes. Good for **write-heavy** workloads, which is why they're the default choice in general-purpose library code: the **Linux kernel's CFS scheduler**, **C++ STL's** `std::map` / `std::set`, and **Java's** `TreeMap` / `TreeSet` are all red-black trees.

Rotations themselves (both structures use the same primitive) restructure a local 3-node neighborhood — a "left rotation" promotes a node's right child to take its place, making the original node the new left child of its former child — to shorten one subtree's height and

lengthen the other's, without breaking binary-search-tree ordering.

14. BFS & DFS

On a tree

- **DFS** goes as deep as possible before backtracking, implemented recursively or with an explicit stack. The three classic orders — **preorder** (visit node, then left, then right), **inorder** (left, node, right — produces sorted output on a BST), and **postorder** (left, right, node) — differ only in *when* the current node is processed relative to its children.
- **BFS** (level-order traversal) visits nodes level by level, implemented with a **queue**: dequeue a node, process it, enqueue its children.

On a graph

The same two shapes apply, but graphs can have cycles, so both need a **visited set** to avoid infinite loops.

- **DFS** (stack or recursion + visited set) explores as far as possible along each branch before backtracking. Uses: detecting cycles, finding connected components, topological sorting (via

reverse postorder), and finding strongly connected components (Tarjan's or Kosaraju's algorithms build directly on DFS).

- **BFS** (queue + visited set) explores all neighbors at the current distance before moving further out. Uses: shortest path in an **unweighted** graph (BFS is guaranteed to find it, since it explores in strictly increasing distance order), bipartiteness checking (2-coloring while traversing), and "shortest number of hops" problems generally.

Both run in $O(V + E)$ time with an adjacency-list representation, since every vertex and every edge is examined a constant number of times.

15. Kahn's Algorithm

Kahn's algorithm produces a **topological sort** of a directed acyclic graph (DAG) — an ordering of vertices such that every edge $u \rightarrow v$ has u appearing before v .

```
compute in-degree[v] for every vertex v
queue = all vertices with in-degree == 0
order = []
```

```
while queue is not empty:
    u = queue.pop()
    order.append(u)
```

```

for each edge (u -> v):
    in-degree[v] -= 1
    if in-degree[v] == 0:
        queue.push(v)

if len(order) < total number of vertices:
    the graph has a cycle – no valid
    topological order exists
else:
    order is a valid topological ordering

```

Why it works: a vertex with in-degree 0 has no unprocessed prerequisite, so it's always safe to place next. Removing it (conceptually, by decrementing its neighbors' in-degrees) can only ever create *new* zero-in-degree vertices, never remove the property from an already-queued one, so the process is safe to run to completion. If it terminates without including every vertex, some remaining vertices must have exclusively formed a cycle among themselves (each still waiting on a prerequisite from within the same unprocessed group) – which doubles as an efficient **cycle-detection** method for free.

Complexity: $O(V + E)$, same as DFS-based topological sort (which instead runs DFS and reverses the postorder finishing times) – the two are duals of each other; Kahn's works forward from sources, DFS-based sort works backward from sinks.

Practical uses: dependency resolution in build systems and package managers (Make, npm, pip figuring out install/build order), course-prerequisite scheduling, resolving formula-evaluation order in a spreadsheet, and CI/CD pipeline stage ordering.

This document is a technical reference compiled for study purposes, drawing on public documentation, research papers (Lamport et al., Castro & Liskov, Flajolet et al.), and open-source implementation details (Redis, Raft, Git).